



Proyecto Final

Sistemas Operativos I

Servidor de eBooks (Calibre-Web)

Inodos y Fragmentación

INTEGRANTES

Diego Corsino
Adriano Barreda
Celeste Vargas
Pathrick Parinango

Ciclo: 2026-I

Fecha: Junio 2026

Índice

1	Introducción	2
2	Arquitectura del despliegue	2
3	Procedimiento	3
4	Instrumentación y experimentos	5
5	Resultados	7
6	Conclusiones	8
7	Anexos	9

1 Introducción

En este informe presentamos el desarrollo y despliegue del **Proyecto 16: Servidor de eBooks (Calibre-Web)**, correspondiente a la evaluación TG04 del curso de Sistemas Operativos I en la Universidad ESAN. El objetivo principal de este trabajo no es simplemente lograr que la aplicación funcione, sino analizar qué sucede “por debajo”, a nivel del kernel de Linux, mientras el contenedor está en ejecución.

El tema de Sistema Operativo protagonista en este proyecto es la **Administración del Sistema de Archivos, Inodos y Fragmentación**. Cuando se manejan bibliotecas grandes con muchos directorios y archivos –como es el caso de Calibre-Web–, el verdadero problema muchas veces no es quedarse sin espacio en el disco duro, sino agotar los inodos (los metadatos del sistema). Para comprobar esto en la práctica, simulamos una carga masiva creando 2,500 libros de prueba. Esto nos permitió evaluar cómo el sistema de archivos de Linux (ext4) asigna los inodos, maneja los bloques y qué tanto se fragmenta la información físicamente.

Para montar el entorno, utilizamos una instancia EC2 en AWS y elegimos **Podman** como motor de contenedores en lugar del clásico Docker. Esta decisión fue clave: al ser *daemonless* (sin un demonio central) y *rootless* (sin permisos de administrador), Podman ejecuta los contenedores directamente bajo nuestro usuario normal (**os_user**). Esto hace que el sistema sea más transparente y nos permite ver de cerca los mecanismos de aislamiento de Linux, como los *Namespaces* para separar permisos (usando **podman unshare**) y los *Control Groups (cgroups)* para limitar cuánta CPU y memoria puede consumir la aplicación.

Finalmente, este proyecto nos permitió llevar a la práctica la teoría vista en clase sobre almacenamiento y recursos. Utilizando herramientas nativas de diagnóstico de Linux (como **df -i**, **stat**, **filefrag** y **podman stats**), y consultando directamente las interfaces del kernel (como **/proc**), pudimos contrastar los conceptos teóricos de los sistemas de archivos con el comportamiento real del servidor.

2 Arquitectura del despliegue

Entorno de Host y Motor de Contenedores

El despliegue se realiza sobre una instancia de **AWS EC2** ejecutando **Ubuntu Linux**. Para aislar la aplicación y mejorar la seguridad del sistema, se utiliza **Podman** como motor de contenedores en modo *rootless* (sin demonio y sin privilegios de superusuario). Todo el entorno se ejecuta dentro del espacio de usuario **os_user**.

Gestión de Volúmenes y Persistencia

Para garantizar la persistencia de la información y separar los datos de la capa efímera del contenedor, se emplean dos volúmenes de tipo *bind mount* con propagación *rprivate*:

- **Volumen de configuración:** el directorio del host `~/proyecto_calibre/config` se mapea a la ruta interna `/config`.
- **Volumen de almacenamiento de libros:** el directorio del host `~/proyecto_calibre/libros` se mapea a la ruta interna `/books`.

Aislamiento y Seguridad (User Namespaces)

El contenedor ejecuta la imagen `linuxserver/calibre-web`. Gracias a la implementación de *user namespaces* en Linux, se logra un aislamiento estricto de identidades:

- El usuario **root** interno del contenedor (que gestiona 8 procesos) nunca es el root real del host; está mapeado al UID 1001 (**os_user**).

- El servicio web interno se ejecuta bajo el usuario **abc**, el cual está mapeado a un UID subordinado (*subuid*) 166446 en la máquina host, minimizando el riesgo de escalado de privilegios.

Control de Recursos (cgroups) y Red

El sistema operativo subyacente restringe el consumo del contenedor mediante *cgroups*, estableciendo un límite máximo de **0.5 CPU** (NanoCpus 5e8) y **512 MB de RAM** (con un swap de 768 MB). En la capa de red, el servicio expone el puerto TCP 8083, el cual está directamente mapeado al puerto 8083 del host para permitir la entrada de tráfico web público.

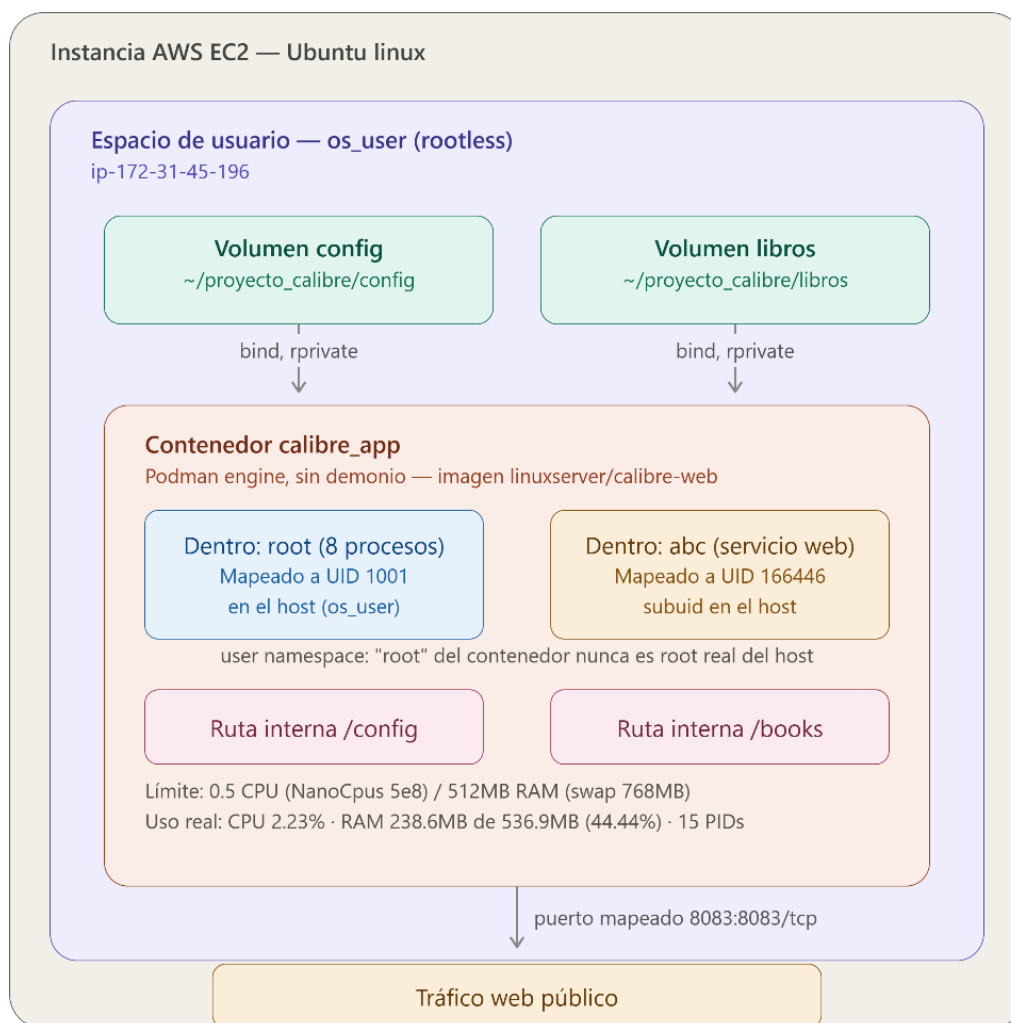


Figura 1: Diagrama de arquitectura del despliegue: host EC2, espacio de usuario rootless, volúmenes persistentes y contenedor **calibre_app** con sus límites de cgroups.

3 Procedimiento

A continuación se documentan, en orden exacto, los comandos utilizados para la instalación y el despliegue. Estos comandos son reproducibles: cualquier persona que los copie en el mismo orden debería poder levantar un entorno idéntico.

1. Empaquetar la imagen en la máquina local (PowerShell)

```
PS C:\Users\...\Escritorio> podman save -o .\calibre_web.tar docker.io/linuxserver/calibre-web
```

Esto genera `calibre_web.tar` en el Escritorio, con la imagen completa (capas + metadatos) lista para transferir.

2. Transferir la imagen al servidor (EC2)

```
scp -i "proyect16.pem" .\calibre_web.tar ubuntu@18.188.134.158:/home/ubuntu/
```

Sube la imagen empaquetada desde la máquina local a la instancia EC2.

3. Mover el archivo al usuario sin privilegios

```
sudo mv /home/ubuntu/calibre_web.tar /home/os_user/
```

Mueve el archivo al directorio del usuario sin privilegios.

4. Asignar permisos al usuario `os_user`

```
sudo chown os_user:os_user /home/os_user/calibre_web.tar
```

Asigna los permisos correctos al usuario `os_user`.

5. Cargar la imagen desde el `.tar`

```
podman load -i ~/calibre_web.tar
```

Carga la imagen de Docker/Podman desde el archivo `.tar`.

6. Crear los directorios para volúmenes persistentes

```
mkdir -p ~/proyecto_calibre/config
```

Crea los directorios para los volúmenes persistentes.

7. Mapear permisos del host al usuario interno del contenedor

```
podman unshare chown -R 911:911 ~/proyecto_calibre/config
```

Mapea los permisos del host al usuario interno del contenedor.

8. Instalar Calibre en el host

```
sudo apt update
sudo apt install -y calibre
```

Instala las herramientas de Calibre en el host para interactuar con la base de datos.

9. Desplegar el contenedor con límites de cgroups

```
podman run -d --name calibre_app -p 8083:8083 \
-v ~/proyecto_calibre/config:/config:Z \
-v ~/proyecto_calibre/libros:/books:Z \
```

```
--memory="512m" --memory-swap="768m" --cpus="0.5" \
docker.io/linuxserver/calibre-web
```

Despliega la aplicación limitando el uso a 0.5 CPUs y 512 MB de RAM mediante *cgroups*.

4 Instrumentación y experimentos

Para auditar el comportamiento del sistema operativo bajo el despliegue de Calibre-Web, realizamos pruebas divididas en tres fases: verificación del aislamiento de procesos, simulación de carga masiva de datos en el sistema de archivos, y comparación del estado físico de los bloques de almacenamiento.

Verificación del aislamiento de procesos

```
podman unshare cat /proc/self/uid_map
```

Con este comando inspeccionamos el mapeo de *namespaces* de usuario (*User Namespaces*). Como se observa en la captura, el sistema traduce dinámicamente nuestro UID del host al usuario interno del contenedor, aislando por completo los privilegios de ejecución sin necesidad de interactuar con el usuario *root*.

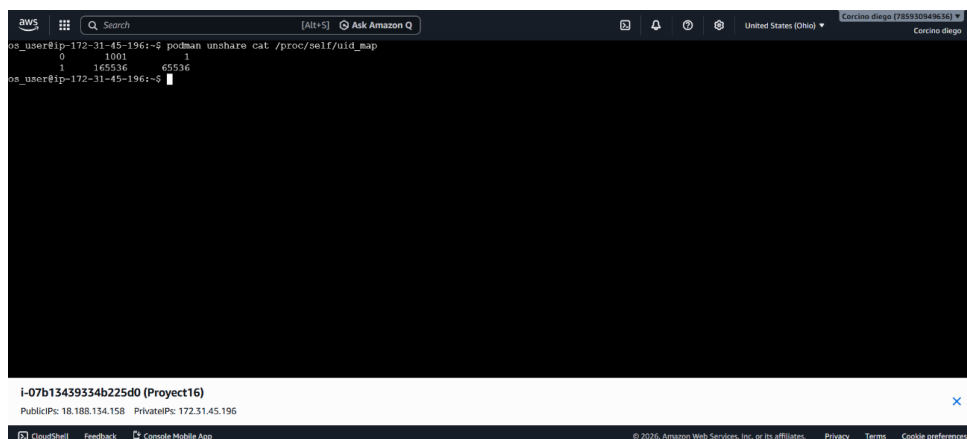


Figura 2: Mapeo de *user namespaces* mediante `podman unshare cat /proc/self/uid_map`.

```
podman stats calibre_app
```

Usamos la telemetría en tiempo real del motor para validar que los límites impuestos por *Control Groups* (*cgroups*) estén activos. La salida demuestra que el contenedor se encuentra limitado a un máximo estricto de 536.9 MB de memoria RAM, registrando un consumo controlado del 43.96% junto al monitoreo activo de procesos hijos (PIDs) y operaciones de entrada/salida (Block IO).

ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET IO	BLOCK IO	PIDS	CPU TIME	AVG CPU %
d82cf9c658dc	calibre_app	0.01%	236MB / 536.9MB	43.96%	198B / 682B	0B / 0B	15	4.893568s	8.87%

Figura 3: Salida de `podman stats calibre_app`: consumo real de CPU, memoria y red bajo los límites de *cgroups*.

Experimento de carga masiva e impacto en inodos

```
df -h ~/proyecto_calibre/libros
```

```
df -i ~/proyecto_calibre/libros
```

Tomamos una medición inicial de la salud de nuestra partición antes de correr la simulación. En este punto inicial, registramos un consumo del 85% de espacio en disco (5.6 GB usados) y únicamente 170,619 inodos ocupados (20% del total de la partición lvm/ext4).

```
os_user@ip-172-31-45-196:~$ df -i ~/proyecto_calibre/libros
Filesystem      Inodes   IUsed   IFree IUse% Mounted on
/dev/root        878080 180565 697515   21% /
os_user@ip-172-31-45-196:~$ df -h proyecto_calibre/libros
Filesystem      Size  Used Avail Use% Mounted on
/dev/root        6.7G  5.6G  1.1G  85% /
os_user@ip-172-31-45-196:~$ |
```

Figura 4: Medición inicial de espacio en disco (df -h) e inodos (df -i) antes de la carga masiva.

```
df -i ~/proyecto_calibre/libros
ls -i
```

Tras ejecutar el script de automatización directamente en la terminal, volvimos a consultar las interfaces de almacenamiento. El resultado empírico demuestra nuestra hipótesis: el espacio libre en gigabytes (df -h) no varió significativamente al tratarse de archivos ligeros, pero los inodos utilizados subieron drásticamente de 170,619 a 180,619 en cuestión de segundos debido a la ramificación de los directorios de Calibre. Con el comando ls -i extrajimos directamente el número identificador único asignado al inodo del primer libro simulado (502026).

```
os_user@ip-172-31-45-196:~$ for i in {1..2500}; do
mkdir -p ~/proyecto_calibre/libros/Autor_Simulado_$i/Libro_Simulado_$i
echo "Contenido de texto indexado del libro numero $i" > ~/proyecto_calibre/libros/Autor_Simulado_$i/Libro_Simulado_$i/Libro_$i.txt
echo "Metadatos XML dummy del libro numero $i" > ~/proyecto_calibre/libros/Autor_Simulado_$i/Libro_Simulado_$i/metadata_$i.opf
done
os_user@ip-172-31-45-196:~$ df -i ~/proyecto_calibre/libros
Filesystem      Inodes   IUsed   IFree IUse% Mounted on
/dev/root        878080 180619 697461   21% /
os_user@ip-172-31-45-196:~$ ls -i ~/proyecto_calibre/libros/Autor_Simulado_1/Libro_Simulado_1/Libro_1.txt
502026 /home/os_user/proyecto_calibre/libros/Autor_Simulado_1/Libro_Simulado_1/Libro_1.txt
os_user@ip-172-31-45-196:~$ |
```

Figura 5: Script de carga masiva (2,500 libros simulados), medición posterior de inodos y consulta del inodo individual con ls -i.

Archivos del sistema vs. archivos subidos por la Web

Analizamos el archivo simulado libro_2119.txt. Al ejecutar stat, el kernel nos devuelve el tamaño físico (50 bytes), los bloques lógicos asignados (8 bloques de E/S de 4096 bytes) y los timestamps exactos de acceso, modificación y nacimiento del archivo creados por nuestro bucle local.

```
os_user@ip-172-31-45-196:~$ stat proyecto_calibre/Libros/Autor_Simulado_2119/Libro_Simulado_2119/libro_2119.txt
File: proyecto_calibre/Libros/Autor_Simulado_2119/Libro_Simulado_2119/libro_2119.txt
size: 50      Blocks: 8      IO Block: 4096   regular file
Device: 259,1 Inode: 510498   Links: 1
Access: (0775/-rwxrwxr-x)  Uid: (166446/ UNKNOWN)   Gid: (166446/ UNKNOWN)
Access: 2026-06-20 21:40:15.532224276 +0000
Modify: 2026-06-20 21:40:15.532224276 +0000
Change: 2026-06-20 22:34:39.015183751 +0000
Birth: 2026-06-20 21:40:15.532224276 +0000
```

Figura 6: Salida de stat sobre el archivo simulado libro_2119.txt.

Repetimos el proceso con el archivo PDF real subido desde el navegador. El comando expone cómo la aplicación web procesó el archivo asignándole el inodo 502004, reservando 280 bloques en disco para sus 143,222 bytes, y registrando la diferencia horaria exacta de sincronización en el atributo Change tras completarse la transferencia por red.

```
os_user@ip-172-31-45-196:~$ stat proyecto_calibre/Libros/'Alessandra Tenorio'/'LECTURA ADICIONAL_EC2_Ensayo (1)'/LECTURA ADICIONAL_EC2_Ensayo - Alessandra Tenorio.pdf
File: proyecto_calibre/Libros/Alessandra Tenorio/LECTURA ADICIONAL_EC2_Ensayo (1)/LECTURA ADICIONAL_EC2_Ensayo - Alessandra Tenorio.pdf
Size: 143222      Blocks: 280      IO Block: 4096   regular file
Device: 259,1    Inode: 502004     Links: 1
Access: (0644/-rw-r--r--)  Uid: (166446/ UNKNOWN)   Gid: (166446/ UNKNOWN)
Access: 2026-06-22 15:14:55.695417835 +0000
Modify: 2026-06-20 22:38:03.506267610 +0000
Change: 2026-06-20 22:38:04.699598581 +0000
 Birth: 2026-06-20 22:38:04.698737829 +0000
```

Figura 7: Salida de `stat` sobre el archivo PDF real subido desde la web.

Evaluamos la fragmentación física en el sistema de archivos para ambos casos. Al usar `filefrag -v`, comprobamos que el subsistema de almacenamiento de Linux mantiene una eficiencia óptima en la asignación de memoria secundaria; tanto el archivo simulado como el documento real subido desde la web muestran exactamente **1 extent found**, lo que significa que fueron guardados de forma contigua en el disco sin fragmentarse.

```
os_user@ip-172-31-45-196:~$ filefrag -v proyecto_calibre/Libros/Autor_Simulado_2119/Libro_Simulado_2119/Libro_2119.txt
Filesystem type is: ef53
File size of proyecto_calibre/Libros/Autor_Simulado_2119/Libro_Simulado_2119/Libro_2119.txt is 50 (1 block of 4096 bytes)
ext:      logical_offset:      physical_offset: length:   expected: flags:
0:      0..      1549556..  1549556:      1:      last, eof
proyecto_calibre/Libros/Autor_Simulado_2119/Libro_Simulado_2119/Libro_2119.txt: 1 extent found
```

Figura 8: `filefrag -v` sobre el archivo simulado: 1 extent found.

```
os_user@ip-172-31-45-196:~$ filefrag -v proyecto_calibre/Libros/'Alessandra Tenorio'/'LECTURA ADICIONAL_EC2_Ensayo (1)'/LECTURA ADICIONAL_EC2_Ensayo - Alessandra Tenorio.pdf
Filesystem type is: ef53
File size of proyecto_calibre/Libros/Alessandra Tenorio/LECTURA ADICIONAL_EC2_Ensayo (1)/LECTURA ADICIONAL_EC2_Ensayo - Alessandra Tenorio.pdf is 143222 (35 blocks of 4096 bytes)
ext:      logical_offset:      physical_offset: length:   expected: flags:
0:      0..      1461001..  1461035:      35:      last, eof
proyecto_calibre/Libros/Alessandra Tenorio/LECTURA ADICIONAL_EC2_Ensayo (1)/LECTURA ADICIONAL_EC2_Ensayo - Alessandra Tenorio.pdf: 1 extent found
```

Figura 9: `filefrag -v` sobre el archivo real subido desde la web: 1 extent found.

5 Resultados

A continuación se consolidan las mediciones obtenidas durante la fase de instrumentación, comparando el estado del sistema de archivos antes y después de la simulación de carga masiva (2,500 libros), así como el comportamiento de los contenedores bajo los límites de cgroups.

Impacto en inodos: antes y después de la carga masiva

Métrica	Antes de la carga	Después de la carga (2,500 libros)
Inodos usados (IUsed)	170,619 (20%)	180,619 (21%)
Espacio en disco usado (df -h)	5.6 GB (85%)	Sin variación significativa (~5.6 GB)
Archivos creados	–	2,500 directorios + 5,000 archivos (texto y metadata)

Tabla 1: Comparación de inodos y espacio en disco antes/después de la carga masiva.

Como se observa, el costo real de la simulación no recayó sobre el espacio en disco, sino sobre la cantidad de inodos disponibles: el consumo de inodos aumentó en 10,000 unidades (170,619 → 180,619) tras crear apenas 2,500 carpetas con archivos de texto livianos, mientras que el uso de espacio en disco (`df -h`) se mantuvo prácticamente igual. Esto confirma empíricamente que, en sistemas con muchos archivos pequeños, el recurso crítico es la tabla de inodos y no la capacidad bruta de almacenamiento.

Comparación: archivo simulado vs. archivo real subido por la Web

Atributo (stat / file-frag)	Archivo simulado	PDF real (Web)
Inodo asignado	510498	502004
Tamaño (bytes)	50	143,222
Bloques de E/S asignados	8 bloques (4096 B)	280 bloques (4096 B)
Extents (filefrag -v)	1 extent found	1 extent found
Fragmentación	Ninguna (contiguo)	Ninguna (contiguo)

Tabla 2: Comparación de atributos de `stat` y `filefrag` entre el archivo simulado (`libro_2119.txt`) y el PDF real subido por la web.

Pese a la diferencia de tamaño (el PDF real pesa cerca de 2,864 veces más que el archivo de texto simulado), ambos archivos obtuvieron exactamente 1 extent found. Esto indica que el sistema de archivos ext4 de la partición aún cuenta con suficiente espacio libre contiguo para asignar bloques de forma secuencial, evitando la fragmentación física incluso bajo la carga generada por nuestra prueba.

Consumo de recursos bajo límites de cgroups

Límite (cgroups)	configurado	Valor máximo	Consumo real (podman stats)
Memoria RAM	536.9 MB	(512m + swap)	236 MB (43.96%)
CPU	0.5 vCPU	(NanoCpus 5e8)	0.01% (en reposo)
Procesos (PIDs)	–		15 PIDs activos

Tabla 3: Límites configurados mediante cgroups frente al consumo real observado con `podman stats`.

La salida de `podman stats` confirma que el kernel respeta estrictamente el límite de memoria definido por cgroups (536.9 MB), incluso cuando el consumo real del contenedor en reposo es bajo (43.96% de memoria y 0.01% de CPU). Esto demuestra que la política de aislamiento funciona como una cota dura: el contenedor podría intentar consumir más recursos, pero el kernel lo restringiría antes de afectar al resto del sistema host.

6 Conclusiones

Aprendizajes del equipo

- El recurso que realmente se agota primero en un servidor de archivos con muchos elementos pequeños no es el espacio en disco, sino los inodos: pasamos de 170,619 a 180,619 inodos usados sin que el porcentaje de disco ocupado se moviera de forma relevante.
- Podman en modo rootless permite observar de primera mano los mecanismos de aislamiento de Linux (*user namespaces*, *subuid/subgid*) sin necesidad de privilegios de superusuario, lo

que hizo tangible la teoría de namespaces vista en clase.

- Las herramientas de diagnóstico nativas de Linux (`df -i`, `stat`, `filefrag`, `/proc`) son suficientes para auditar el comportamiento real del kernel bajo un contenedor, sin necesidad de instrumentación adicional o herramientas de terceros.
- Los cgroups configurados (`-memory`, `-memory-swap`, `-cpus`) actúan como una cota dura impuesta por el kernel, lo cual confirma en la práctica que la planificación y administración de recursos no depende de la buena voluntad de la aplicación, sino del sistema operativo subyacente.

Cuellos de botella identificados

El principal cuello de botella observado durante las pruebas fue la disponibilidad de inodos y no la capacidad de almacenamiento bruta. Al tratarse de una partición ext4 con un número fijo de inodos reservados desde su formateo, una biblioteca con un volumen muy alto de archivos pequeños (como portadas, metadatos `.opf` y fragmentos de texto por libro) puede agotar la tabla de inodos mucho antes de llenar el disco físicamente, dejando el sistema incapaz de crear nuevos archivos a pesar de mostrar espacio libre en GB. En segundo lugar, el límite de 0.5 CPU impuesto por cgroups representa un techo explícito para escenarios de mayor concurrencia (por ejemplo, varias conversiones de formato de libros en simultáneo), aunque durante nuestras pruebas el contenedor permaneció lejos de ese límite al no generarse carga de CPU sostenida.

Mejoras propuestas

- Formatear la partición de almacenamiento de libros con un mayor número de inodos por bloque (parámetro `-i` de `mkfs.ext4`), o evaluar sistemas de archivos como XFS, más flexibles para escenarios con gran cantidad de archivos pequeños.
- Repetir el experimento generando carga de CPU sostenida (por ejemplo, conversiones masivas de formato con Calibre) para observar cómo el planificador del kernel distribuye el 0.5 CPU asignado y si aparece *throttling* visible en `podman stats`.
- Automatizar la recolección de métricas (`df -i`, `podman stats`) mediante un script que registre el historial en el tiempo, en lugar de capturas puntuales, para visualizar tendencias de consumo de inodos a mayor escala (por ejemplo, 25,000 o 50,000 libros).
- Migrar el despliegue a un Pod de Podman con múltiples contenedores (por ejemplo, separando el servicio web de un proceso de indexación en segundo plano), aprovechando la red compartida del pod para evaluar el aislamiento de recursos entre procesos relacionados.

7 Anexos

- **Video de la demo (3–5 minutos):** <https://drive.google.com/file/d/1ayskWalWyM9VvKa7YIwmAdNYPy/view?usp=sharing>
- **Repositorio del proyecto:** <https://github.com/IDk-jg/Proyecto-Podman-Calibre-Web.git>

Referencias bibliográficas

- linuxserver/calibre-web. (s.f.). *Documentación oficial de la imagen Docker/Podman*. <https://hub.docker.com/r/linuxserver/calibre-web>
- Podman.io. (s.f.). *Documentación oficial de Podman – Rootless containers y namespaces*. <https://docs.podman.io/en/stable/markdown/podman.1.html>

- Documentación del kernel de Linux. (s.f.). *Sistema de archivos /proc*. <https://www.kernel.org/doc/Documentation/filesystems/proc.rst>
- man7.org. (s.f.). Páginas de manual: <https://man7.org/linux/man-pages/man1/df.1.html> (df), <https://www.man7.org/linux/man-pages/man1/stat.1.html> (stat), <https://man7.org/linux/man-pages/man8/filefrag.8.html> (filefrag), <https://man7.org/linux/man-pages/man1/unshare.1.html> (unshare).
- Material de clase del curso de Sistemas Operativos I, Universidad ESAN, ciclo 2026-I (notas y diapositivas de gestión de sistemas de archivos, inodos y cgroups).

Imágenes base citadas

Las siguientes diapositivas de clase fueron utilizadas como material de referencia teórica, tal como exige la política de originalidad del curso.

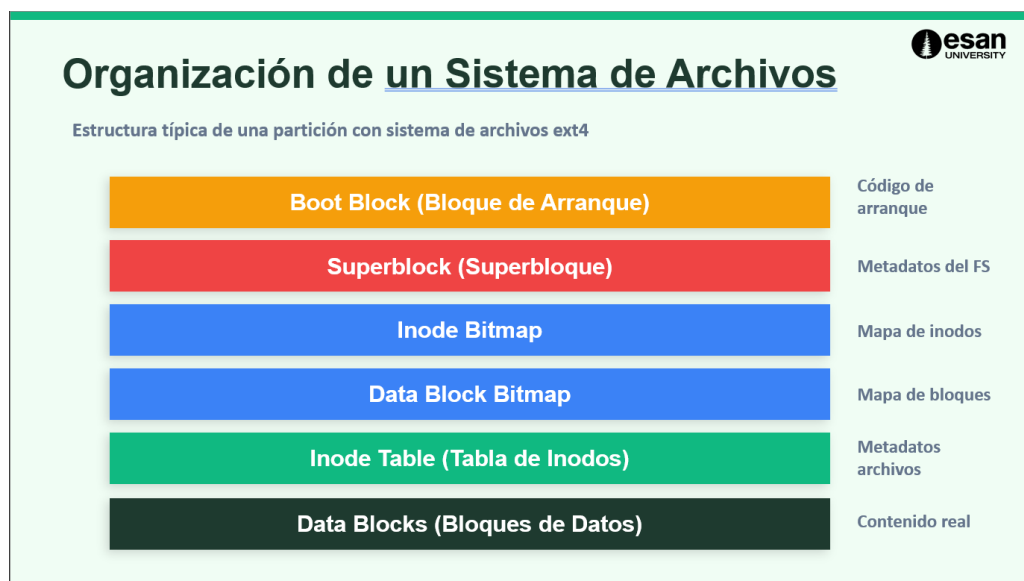
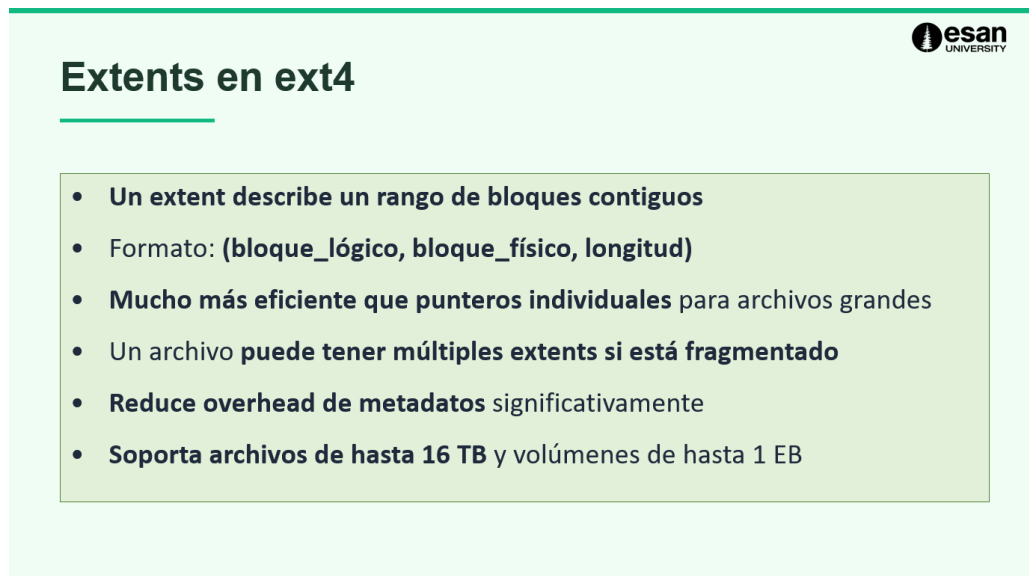


Figura 10: Organización de un sistema de archivos ext4. Fuente: material de clase, Universidad ESAN, ciclo 2026-I.



Figura 11: Estructura de un inodo. Fuente: material de clase, Universidad ESAN, ciclo 2026-I.



Extents en ext4

- **Un extent describe un rango de bloques contiguos**
- **Formato: (bloque_lógico, bloque_físico, longitud)**
- **Mucho más eficiente que punteros individuales** para archivos grandes
- Un archivo **puede tener múltiples extents si está fragmentado**
- **Reduce overhead de metadatos** significativamente
- **Soporta archivos de hasta 16 TB** y volúmenes de hasta 1 EB

Figura 12: Extents en ext4. Fuente: material de clase, Universidad ESAN, ciclo 2026-I.